Université de Limoges — Faculté des Sciences & Techniques

Faculté des Sciences et Techniques de Limoges

L2 - Bases du langage C
TP1 - Premiers pas en C

Maxime MARIA

Respectez bien les consignes et testez le bon fonctionnement de tous vos programmes ! Pensez aussi à sauvegarder régulièrement ainsi qu'à garder tous vos codes !

1 Hello World !

Dans ce premier exercice, nous allons simplement compiler et exécuter le code suivant :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(int argc, char *argv[])
5 {
6     printf("Hello World!");
7
8     return EXIT_SUCCESS;
9 }
```

Les deux premières lignes permettent d'inclure des bibliothèques (*cf.* cours). La ligne 4 déclare la fonction principale du programme (`main`) dont le prototype est toujours :

```
int main(int argc, char *argv[])
```

Nous reviendrons sur ses arguments dans l'exercice 2. Pour le moment, remarquez seulement que cette fonction retourne un entier (`int`). Cet entier correspond à un « code d'erreur ». Ici, nous retournons `EXIT_SUCCESS` *i.e.* « tout s'est bien déroulé » (ligne 8). Cette constante est définie dans la bibliothèque `<stdlib.h>` et vaut `0`.

> Par convention, un programme C doit retourner 0 s'il s'est exécuté sans erreur et un entier non nul sinon. Les codes d'erreurs compris entre 126 et 255 sont réservés par le système et décrivent des erreurs spécifiques. Par exemple, le code `139` informe d'une erreur d'accès mémoire (*segmentation fault*). Vous le verrez souvent celui-là ! Pour retourner un code d'erreur par défaut, vous pouvez utiliser `EXIT_FAILURE` de `<stdlib.h>`.

1. Créez un fichier `hello_world.c` dans le dossier de votre choix (*e.g.* `TP_Ex1`) et recopiez-y le code ci-dessus.
2. Ouvrez le terminal (la console) et placez-vous dans le dossier de votre programme. Compilez le programme via la ligne de commande :

   ```
   gcc -Wall hello_world.c -o Hello
   ```

   Décrivons cette ligne :
   - `gcc` : appel du compilateur
   - `-Wall` : option du compilateur pour afficher tous les messages d'avertissement (*Warning All*)
   - `hello_world.c` : le nom du fichier à compiler
   - `-o Hello` : `-o` permet de spécifier le nom du fichier exécutable (ici `Hello`)
3. Si tout a bien fonctionné, un fichier exécutable a dû être créé dans le dossier de votre programme (`Hello` sous Linux et `Hello.exe` sous Windows). Exécutez-le via la ligne de commande : `./Hello`
   Normalement, `Hello World!` s'est affiché dans la console.

2 Les arguments de `main`

Les deux arguments de la fonction `main` sont :

- `int argc` : le nombre de paramètres passés au programme au moment de son exécution ;
- `char *argv[]` : un tableau de chaînes de caractères contenant les paramètres passés au programme au moment de son exécution.

Autrement dit, `argv` est un tableau contenant `argc` arguments stockés sous la forme de chaînes de caractères.

> Le premier élément du tableau (`argv[0]`) contient toujours le chemin de votre programme (dans l'exercice précédent, il s'agissait donc de `Chemin_du_dossier_du_programme/Hello`). Par conséquent, `argc` est toujours au moins égal à 1.

1. Modifiez le programme précédent afin d'afficher `Hello XXX!` où `XXX` est une chaîne de caractères passée en paramètres lors de l'exécution de votre programme. Pour passer un paramètre à votre programme, il suffit de l'écrire après la commande d'exécution. Par exemple, si votre programme s'appelle « Hello » et que vous voulez lui passer l'argument « Maxime », vous écrirez : `./Hello Maxime`. Pour plus d'informations sur la fonction `printf`, consultez le cours ou `https://devdocs.io/c/io/fprintf` (ne vous souciez pas de la valeur de retour).
2. Exécutez votre programme sans donner de paramètre. Que se passe-t-il ? Modifiez votre programme pour afficher un message d'erreur et retourner `EXIT_FAILURE` si le nombre de paramètres donnés n'est pas valide.
3. Modifiez une nouvelle fois votre programme pour qu'il prenne en compte deux paramètres, un nom et un âge, et affiche `Hello XXX qui a YY ans!`. Vous utiliserez deux variables temporaires pour stocker le nom (`char *`) et l'âge (`int`) avant de les afficher. Le problème est que les paramètres du programme sont transmis sous la forme de chaîne de caractères... Il va donc falloir convertir le paramètre contenant l'âge en entier via la fonction `atoi` : `https://devdocs.io/c/string/byte/atoi` (*Ascii TO Integer*).

3 Découper son programme en plusieurs fichiers

Pour plus de lisibilité et pour favoriser la réutilisation de votre code, il convient d'organiser son code en regroupant les fonctions dans des fichiers séparés : on parle alors de bibliothèques. Par exemple, `<stdio.h>` et `<stdlib.h>` (incluses dans l'exercice précédent) font partie de bibliothèque standard du langage C.

Pour créer vos propres bibliothèques (dites locales), vous allez devoir créer de nouveaux fichiers, les compiler et les lier à votre programme. Pour cela, il faut :

- créer un fichier `.h` contenant la déclaration des fonctions (*i.e.* seulement leur prototype) ;
- créer un fichier `.c` (du même nom) contenant l'implémentation des fonctions (prototype + corps) ;
- inclure la bibliothèque dans le fichier où elle sera utilisée (`#include "nom_bibliotheque.h"`) ;
- compiler en précisant la bibliothèque utilisée : `gcc -Wall programme.c nom_bibliotheque.c -o...`

> L'inclusion des bibliothèques locales se fait en utilisant des guillemets (`#include "ma_bib.h"`) et non avec des chevrons (`#include <stdio.h>`).

1. Créez une bibliothèque nommée `salutation` contenant une fonction `salut` qui prend en paramètre une chaîne de caractères et réalise le même affichage que la question 1 de l'exercice 2. Testez son fonctionnement.
2. Ajoutez une fonction similaire, prenant un entier supplémentaire en paramètre et réalisant le même affichage que la question 3 de l'exercice 2. Testez son fonctionnement.

> Contrairement à Processing par exemple, il n'y a pas de notion de surcharge de fonction en C, il faut donc que les procédures et fonctions aient toutes des noms différents !

4 Les bases de l'écriture

Dans cet exercice, vous allez réaliser un programme qui affiche un nombre dans différentes bases.

1. Créez un nouveau programme qui prend en paramètre un entier non signé (`unsigned int`) et l'affiche en base 8 et 16. Pour cela, il suffit simplement d'utiliser le bon caractère de formatage après `%` dans la fonction `printf`. Par exemple, `79` donnera `117` en base 8 et `4F` en base 16. Nous supposerons que l'argument fourni par l'utilisateur à l'exécution du programme entre un entier positif valide, codable sur 32 bits.
2. C ne propose pas de formatage en binaire (base 2). Écrivez alors une procédure `printBinary` prenant un entier non signé `N` comme argument et l'affichant en binaire avec le bit de poids fort à gauche et un espace entre chaque chiffre. Par exemple, `79` donnera `1 0 0 1 1 1 1`.

   L'algorithme 1 une procédure en pseudo-code réalisant cette opération sans utiliser de tableau, ni d'opérateur bit à bit, ni de récursivité :

Algorithm 1 Procédure affichant un entier non signé en binaire, avec le bit de poids fort à gauche.

```
procedure PRINTBINARY(n)                          ▷ n ≥ 0 : number to print in binary.
    p ← 1
    ▷ Find the power of 2 closest to n.
    while p ≤ n/2 do
        p ← p × 2
    end while
    ▷ Print each bit.
    while p > 0 do
        print(n/p)          ▷ Print the quotient of the euclidean division : 0 or 1.
        n ← n  mod p                                  ▷ Keep the remainder.
        p ← p/2                                       ▷ Divide p to get the next bit.
    end while
end procedure
```

3. Testez votre programme avec différentes valeurs, dont 0 et 4294967295 (plus grand entier non signé sur 32 bits), et vérifiez son bon fonctionnement. Corrigez les erreurs si nécessaire (pensez bien que nous traitons des entiers non signés).
4. *(Optionnel)* Écrivez une procédure `printBinaryRec` qui réalise l'affichage binaire de façon récursive.
5. *(Optionnel)* Modifiez l'affichage de votre programme pour que les différentes bases soient parfaitement alignées. Par exemple, pour `79`, l'affichage donnera :

```
79 en base 2  : 1 0 0 1 1 1 1
79 en base 8  : 1 1 7
79 en base 16 : 4 F
```

Indice : renseignez-vous sur la fonction `sprintf` (`https://devdocs.io/c/io/fprintf`).